Pay1oad 악성코드 스터디

악성코드 분석 스터디 6주차 “Reversing Literacy 01”

CONTENTS

01 • 왜 리버싱은 어려운가?
- 리버싱의 이해

02 • 리버싱의 여러 접근 관점
- 악성코드 분석, 암호해제

03 • 리버싱의 기본적인 구조
- 컴퓨터구조, 스택

04 • C언어와 어셈블리어
- 문법과 번역되는 언어의 이해

05 • Q&A

Chapter. • 1

왜 리버싱은 어려운가?

왜 리버싱은 어려운가?

프로그래밍 언어들

프로그래밍 언어, 라이브러리, 플랫폼등 다양한 변수로
모든 프로그램을 분석한다면 배워야할건 매우 많습니다.

리버싱은 무엇인가?

```c
#include <stdio.h>
int main(void)
{
  printf("Hello World");
}
```

```
#include <stdio.h>

int main()
{
00401080 55                  push  ebp
00401081 8B EC               mov   ebp,esp
  printf("Hello World");
00401083 68 20 30 40 00      push   403020h
00401088 E8 B3 FF FF FF      call   printf (0401040h)
0040108D 83 C4 04            add    esp,4
}
00401090 33 C0               xor    eax,eax
00401092 5D                  pop    ebp
00401093 C3                  ret
```

다양한 언어에 따라 어셈블리어를 읽기위해서는 최소한의 개발 지식이 있어야 읽기가 편합니다.

리버싱은 무엇인가?

```cpp
#include <cstdio>

class A1
{
  A1()
  {
    printf("A1 생성자\n");
  }
  ~A1()
  {
    printf("A1 소멸자\n");
  }
};

int main()
{
  A1 a;
}
```

```
int main()
{
004010C0  push    ebp
004010C1  mov     ebp,esp
004010C3  push    ecx
  A1 a;
004010C4  lea     ecx,[a]
004010C7  call    A1::A1 (0401080h)
}
004010CC  lea     ecx,[a]
004010CF  call    A1::~A1 (04010A0h)
004010D4  xor     eax,eax
004010D6  mov     esp,ebp
004010D8  pop     ebp
004010D9  ret
```

다양한 언어에 따라 어셈블리어를 읽기위해서는 최소한의 개발 지식이 있어야 읽기가 편합니다.

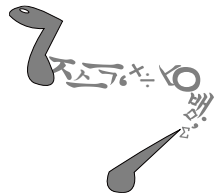
리버싱은 무엇인가?

```
Console.WriteLine("Hello, World!");
```

→

```
ldstr     "Hello, World!"
call      void System.Console::WriteLine(string)
nop
ret
```

다양한 언어에 따라 어셈블리어를 읽기위해서는 최소한의 개발 지식이 있어야 읽기가 편합니다.



리버싱은 무엇인가?

```c
#include <stdio.h>
int main(void)
{
  printf("Hello World");
}
```

```
#include <stdio.h>

int main()
{
00401080 55                  push  ebp
00401081 8B EC               mov   ebp,esp
  printf("Hello World");
00401083 68 20 30 40 00      push   403020h
00401088 E8 B3 FF FF FF      call   printf (0401040h)
0040108D 83 C4 04            add    esp,4
}
00401090 33 C0               xor    eax,eax
00401092 5D                  pop    ebp
00401093 C3                  ret
```

다양한 언어에 따라 어셈블리어를 읽기위해서는 최소한의 개발 지식이 있어야 읽기가 편합니다.

리버싱은 무엇인가?

```c
#include <stdio.h>
int main(void)
{
  printf("Hello World");
}
```

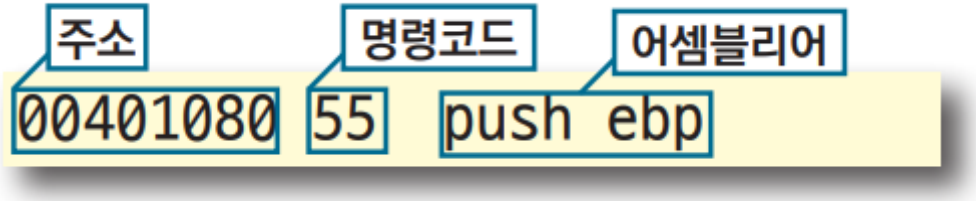
```
#include <stdio.h>

int main()
{
00401080 55                push  ebp
00401081 8B EC             mov   ebp,esp
  printf("Hello World");
00401083 68 20 30 40 00    push   403020h
00401088 E8 B3 FF FF FF    call   printf (0401040h)
0040108D 83 C4 04          add    esp,4
}
00401090 33 C0             xor    eax,eax
00401092 5D                pop    ebp
00401093 C3                ret
```

```c
#include <stdio.h>
int main(void)
{
  printf("Hello World");
}
```

```
{
 fp,lr,[sp,#-0x10]!
fp,sp
printf("Hello World!");
adrp        x8,¦$SG4294967289¦
add         x0,x8,¦$SG4294967289¦
bl          printf
}
mov         w0,#0
ldp         fp,lr,[sp],#0x10
ret
```

같은 C언어코드라도 아키텍쳐에 따라 결과물을 알려면 또 공부해야한다.

기본적인 어셈블리어 구조



위 구조가 일반적인 어셈블리어의 기본 골격입니다.

악성코드 분석 능력 검증 체크리스트

[악성파일 세부분석]	악성 파일의 해시값을 구할 수 있는가?
	레지스트리(자동실행, 스케줄링 등)를 분석할 수 있는가?
	환경에 맞는 도구를 사용하여 분석할 수 있는가? (IDA, x64dbg, Process Monitor, API Monitor, Autoruns 등)
	샌드박스 분석 VM 사이트 사용할 수 있는가?
	악성 파일 실행 시 추가로 생성되는 파일을 확인할 수 있는가?
	악성 파일의 주요 행위를 분석할 수 있는가?
	악성코드의 유형별 분류를 할 수 있는가?
	악성코드에 적용된 안티디버깅 기법을 식별할 수 있는가?
	문서형 악성코드를 분석할 수 있는가?

기본적인 체크리스트만 한다해도 그 양이 몇 달은 공부해야합니다.

악성코드 분석 능력 검증 체크리스트

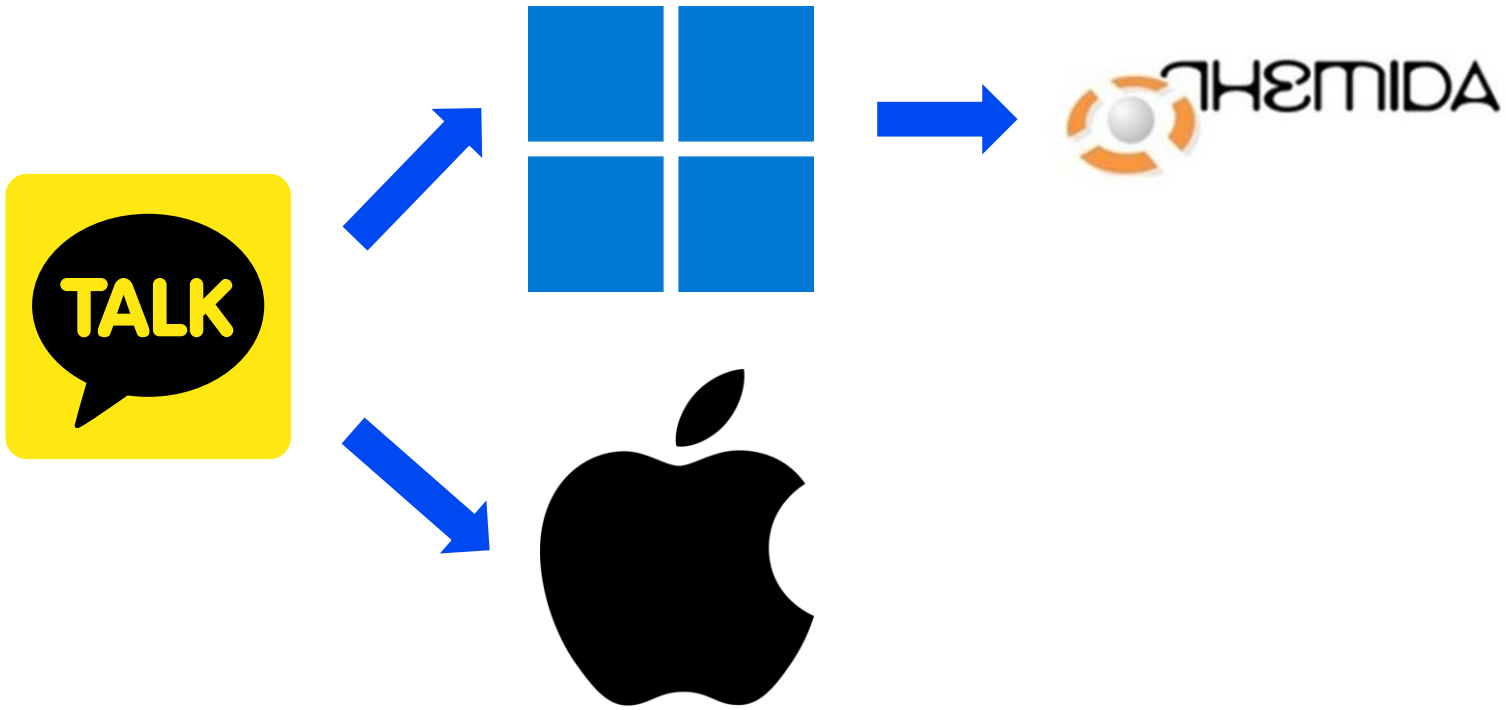
[윈도우 아키텍처]	유저모드와 커널모드의 차이점
	프로세서의 의미
	윈도우 서비스의 이해
	윈도우 이벤트 로그의 이해
	윈도우 레지스트리의 이해
	MBR과 GPT의 차이
	윈도우 계정과 SID
	Windows Sysinternals 도구들의 사용법을 숙지하고있다.
	그외 웹 로그 및 간단한 로그 포랜식을 수행할 수 있다.

기본적인 체크리스트만 한다해도 그 양이 몇 달은 공부해야합니다.

Chapter. • 2

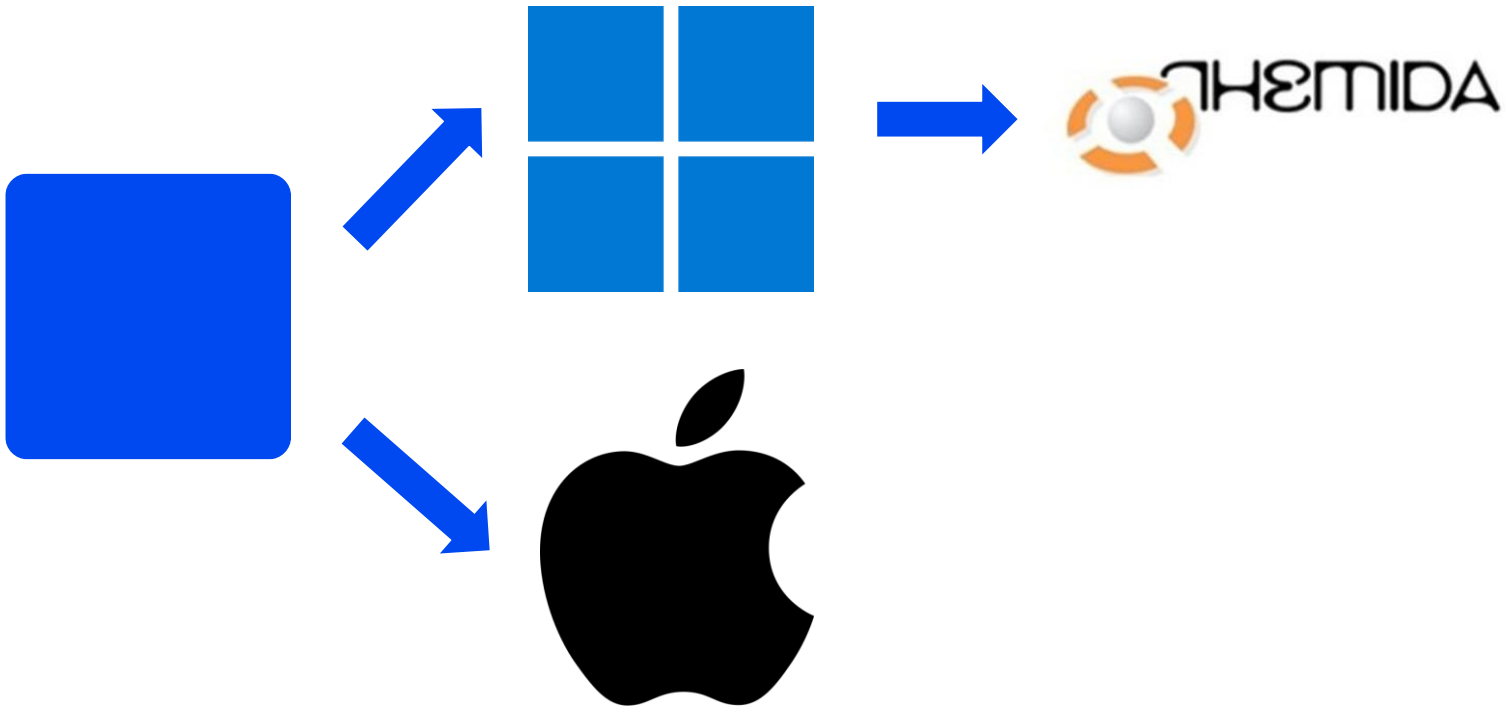
리버싱의 여러 접근 관점

접근 시나리오 예



예를 들어 카카오톡의 채팅을 복원하고 싶다.
카카오톡은 카톡 내용을 암호화하네
그리고 윈도우는 더미다라는 보호기법을 쓰네
차다리 애플정책으로 그런 보호를 못하는 곳에 가서
리버싱하는게 상대적으로 쉽겠다.

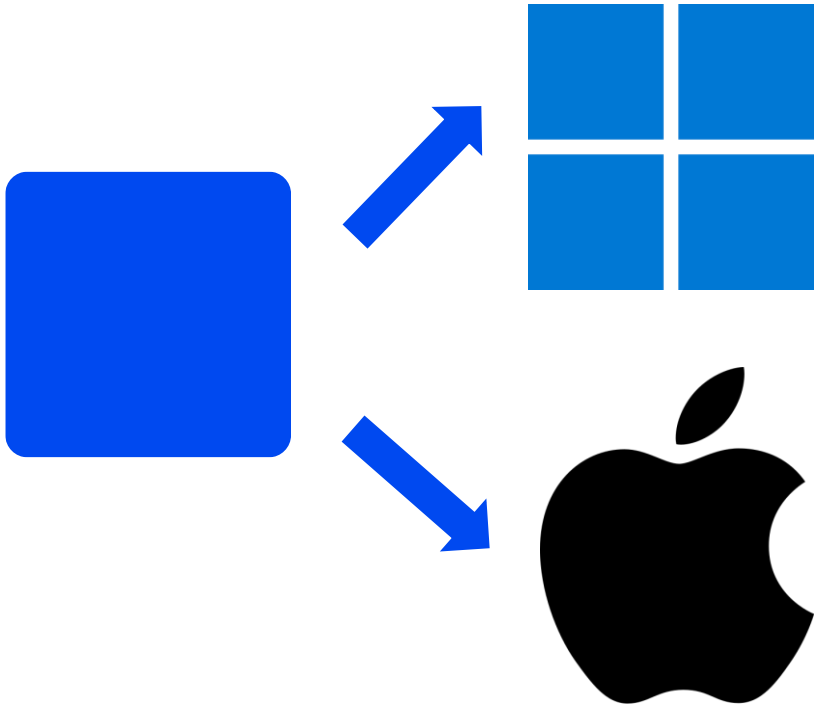
접근 시나리오 예2



예를 들어 어떤 상용프로그램의 라이선스을 해킹하고 싶다.

그러면 라이선스 관련된 코드를 이해하고
라이브러리와 개발을 이해하고 분석하고
그 부분만 노리면 되겠네!

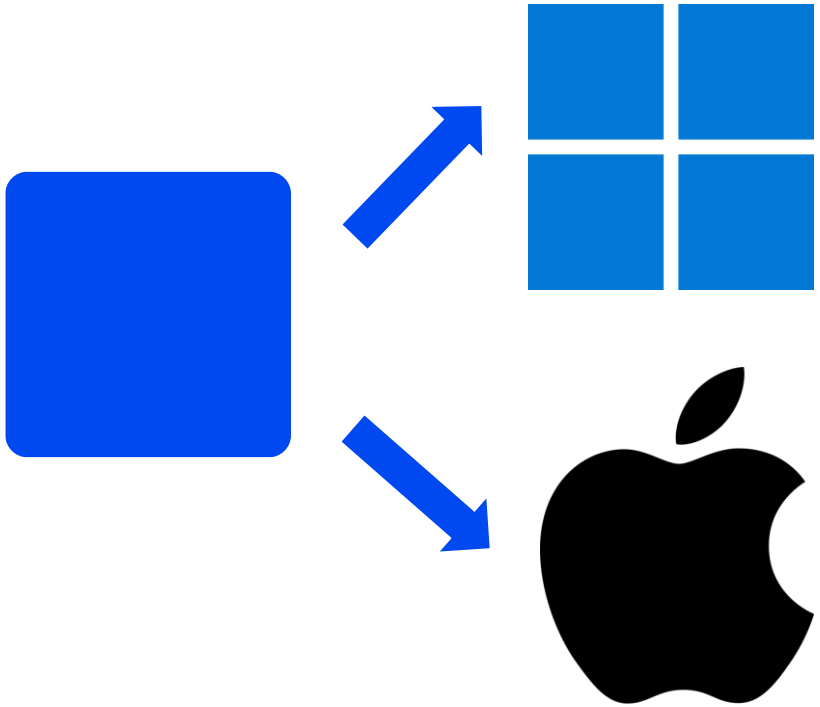
공부의 방법



그러니까 리버싱을 다 배우것은 수학의 모든 과목을 배우는 것랑 같습니다.

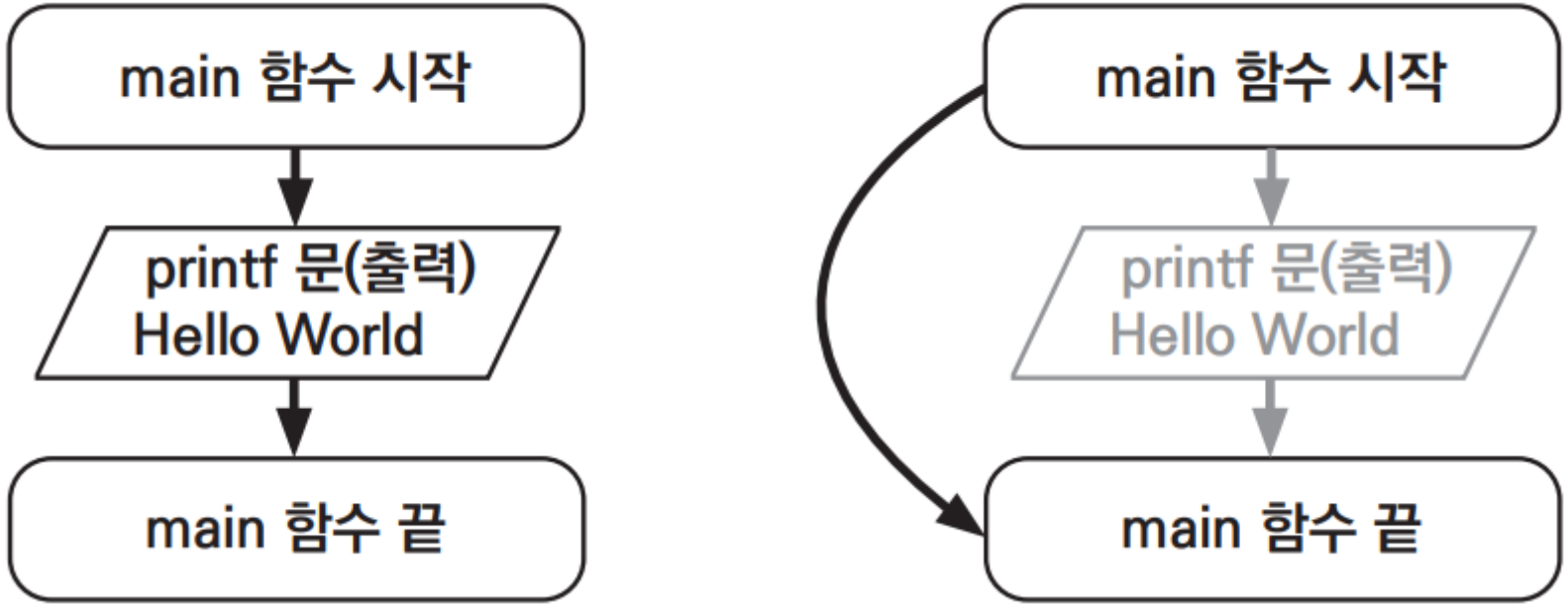
어느정도 주체적인 목적을 갖고 **목표를 위한 수단으로** 보면
조금 더 명확하게 공부할 주제가 나오니 그렇게 **하나씩 늘려가는 걸로**
리버싱을 공부하는 걸 추천합니다.

공부의 방법 (악성코드)



윈도우 악성코드면
악성코드의 기능을 쪼개서 그것부터 보는걸 추천합니다.
악성 행위가 무엇일까
파일을 암호화 하는것, 사용자 PC에 파일 훔치기, 광고 띄우기, 키보드 입력 훔치기등등 방법에 따른 다양한 개발은 여전히 많으니 줄여서 공부해야 편합니다.

리버싱이란

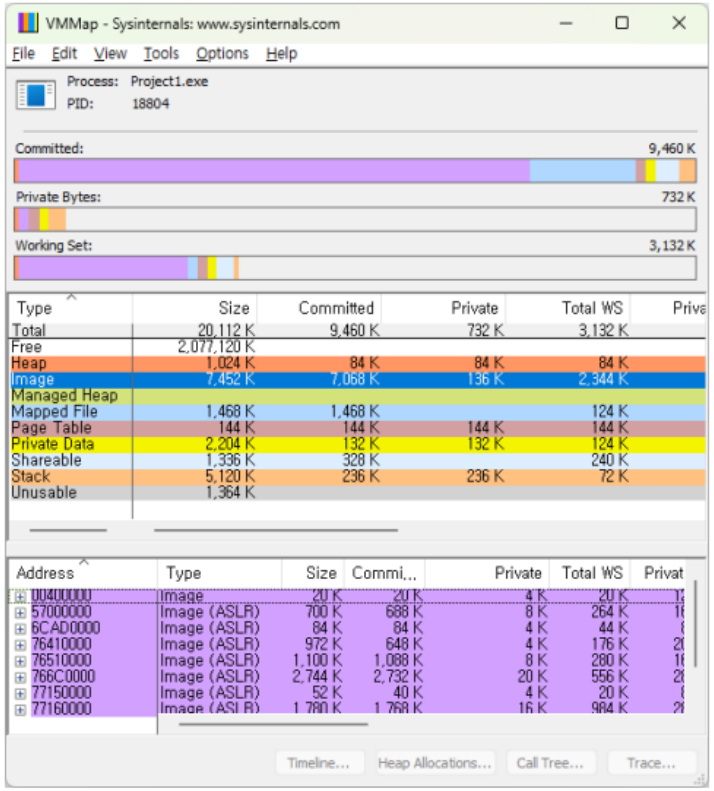


위 그림 처럼 자신이 이해한다면 순서를 바꾸어서 로직을 바꿀 수도 있고 이해한 내용을 그대로 활용할 수 있는 학문입니다.

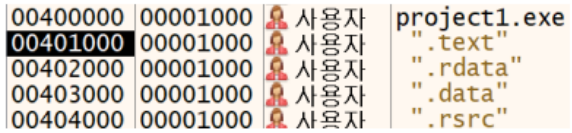
Chapter. • 3

리버싱의 기본적인 구조

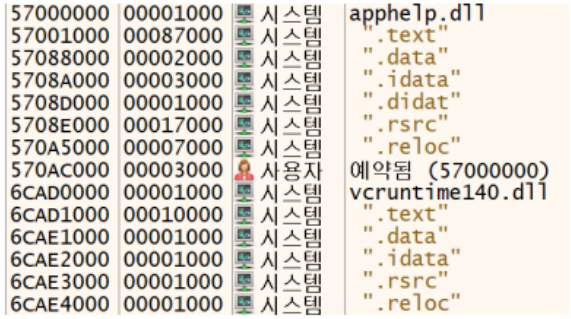
리버싱의 기본 구조



▲ VMMap



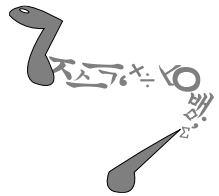
▲ x64dbg 메모리 맵



▲ x64dbg 메모리 맵

우리가 실행하는 프로그램은 RAM에 올라간다 알려져있지만 실제로는 꽤나 복잡한 구조입니다.

가상 메모리라는 걸 사용하여 메모리의 용량을 벗어나도 실행되게하는 기능을 사용하는걸 기억하시면 됩니다.



리버싱의 기본 구조

```
00401080   55            push ebp                      main.c:3
00401081   8BEC          mov ebp,esp
00401083   68 88304000   push project1.403088          main.c:4, 403088:"Hello World"
00401088   E8 B3FFFFFF   call <project1._printf>
0040108D   83C4 04       add esp,4
00401090   33C0          xor eax,eax                   main.c:5
00401092   5D            pop ebp
00401093   C3            ret
```

▲ x64dbg CPU

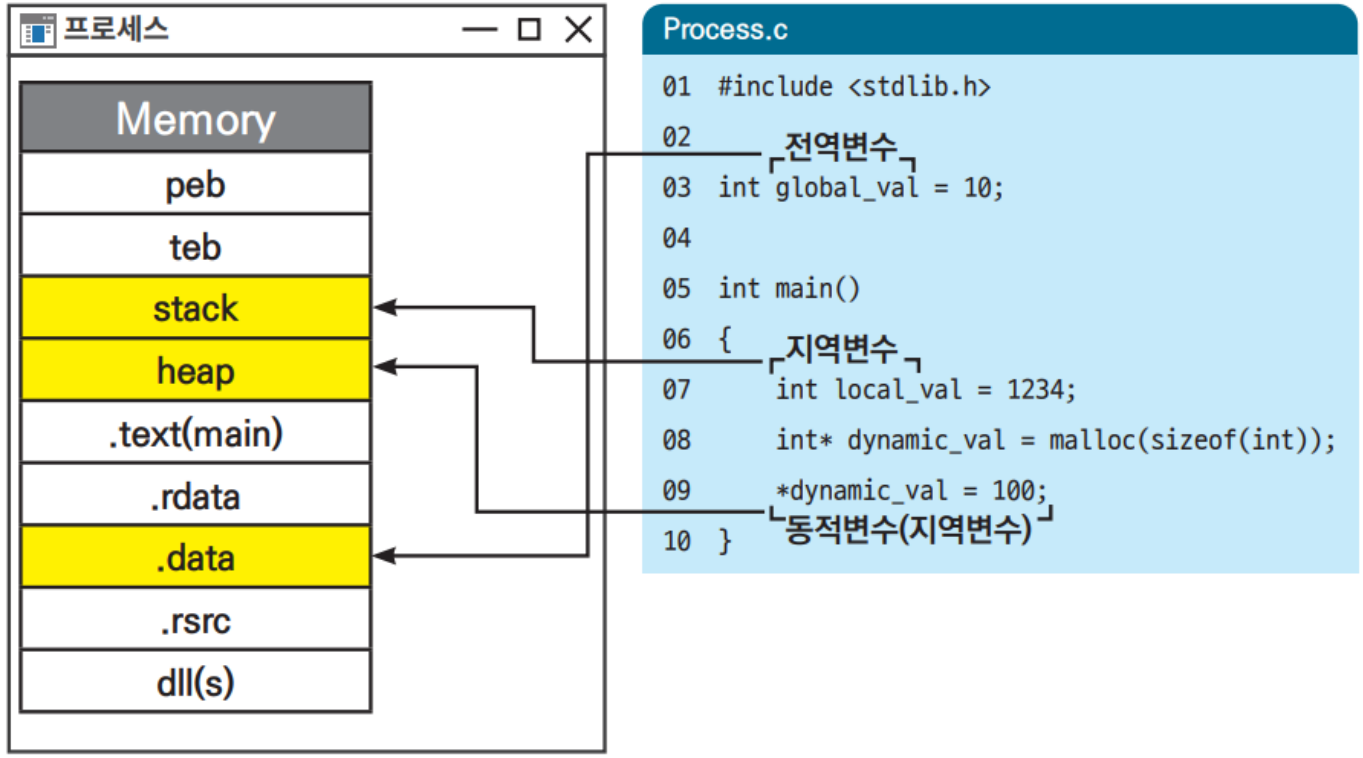
덤프 1 | 덤프 2 | 덤프 3 | 덤프 4 | 덤프 5 | 주시 1 | [x=] 로컬 | 구조체 | 디스어셈블리

주소	Hex												ASCII
00401080	55	8B	EC	68	88	30	40	00	E8	B3	FF	FF	FF 83 C4 04	U.ìh.0@.è³ÿÿÿ.Ä.
00401090	33	C0	5D	C3	56	6A	01	E8	D0	0C	00	00	E8 33 05 00	3À]ÃVj.èÐ...è3..

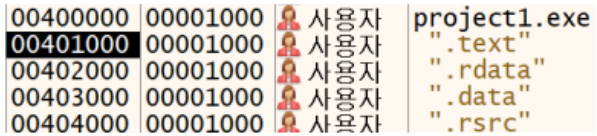
▲ x64dbg 덤프

어셈블리어는 사실 기계어의 나열입니다.
보통 0과1이 아닌 16진수로 보고 그걸 해석하면
어셈블리어가 나옵니다.

프로세스란



▲ 디버깅중 상단메뉴 아래



▲ x64dbg 메모리 맵

프로세스는 여러 구역으로 나뉘어있고 그 구역들에 맞게 코드가 맵핑된다고 생각하면 됩니다.

리버싱의 기본 구조

Little Endian

0x78	0x56	0x34	0x12

Big Endian

0x12	0x34	0x56	0x78

Int를 저장할때 CPU에 따라
Little Endian과 Big Endian으로 두가지 방식으로 저장합니다.
12 34 56 78 정순서를
Big Endian은 78 56 34 12로 저장합니다.

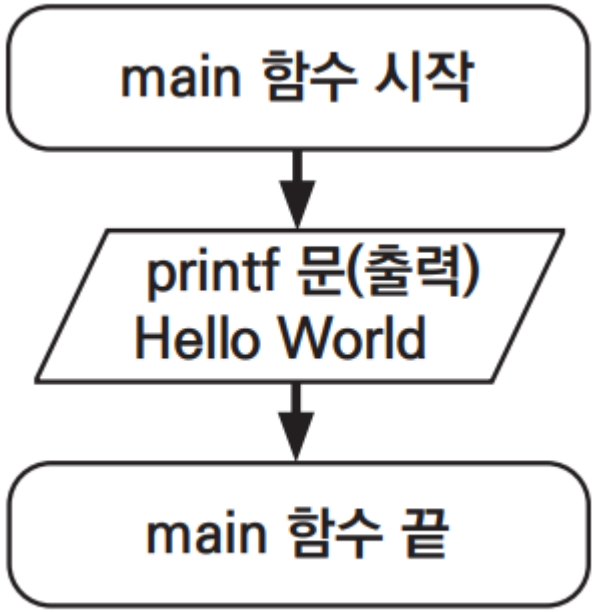
Chapter. • 3

C언어와 어셈블리어

코드를 이해하는 방법 (도식화)

Hello_World.c

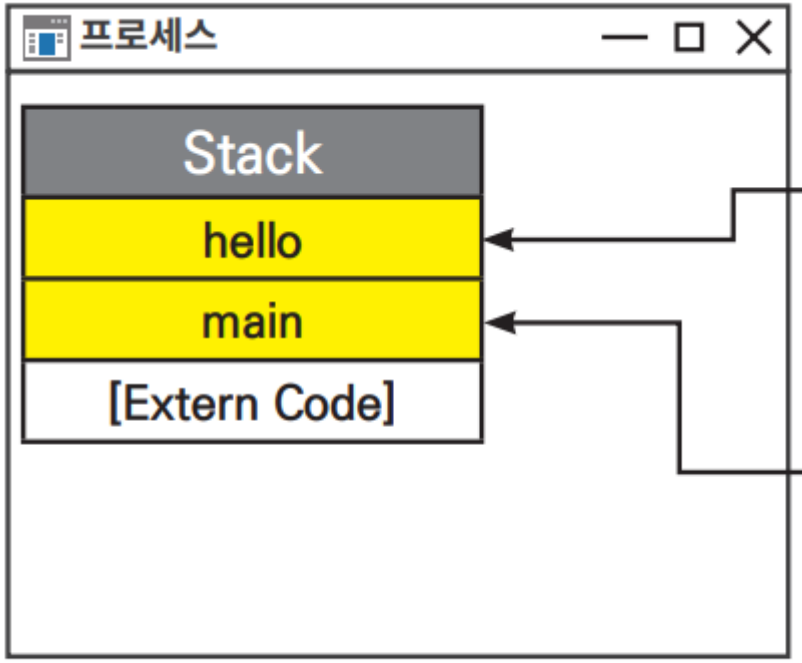
```
01  #include <stdio.h>
02
03  int main()
04  {
05      printf("Hello World");
06  }
```



▲ 정상적인 흐름의 main 함수 순서도

코드를 공부하는 방법중 하나인 도식화가 있습니다.
사실 요즘 와서는 간단하게는 도식화 없이 하지만
그러한 도식화가 어셈블리어에서는 되게 도움이 됩니다.

함수를 이해하는 방법 (함수스택)



Function_Basic_Structure.c

```
01  #include <stdio.h>
02
03  void hello() {
04      printf("hello world\n");
05  }
06
07  int main() {
08      hello();
09  }
```

hello함수

메인함수

함수 또한 스택처럼 삶입니다.(C/C++ 기준)
위 개념을 기억하면 여러가지로 활용할 수 있습니다.
가장 밑단에 함수를 찾아서 역으로 추적하는 BackTrace
등 활용이 가능합니다.

mov 어셈블리어

```
출저: Intel® 64 and IA-32 Architectures Software Developer Manuals / 5.1.1 Data Transfer Instructions

MOV
Move data between general-purpose registers;                          범용 레지스터 간 데이터 이동;
move data between memory and generalpurpose or segment registers;     메모리와 범용 또는 세그먼트 레지스터 간 데이터 이동;
move immediates to general-purpose registers.                         즉시값을 범용 레지스터로 이동.

mov a, b                  // 뜻 a에 b의 값을 대입한다.
mov dword ptr [var],0Ah   // 뜻: var에 10(0Ah)로 대입한다.
mov eax,dword ptr [var]   // 뜻: eax에 var의 값에 대입한다.
neg eax                   // 뜻: eax의 값을 음수로 하고 대입한다.
mov dword ptr [var],eax   // 뜻: var에 eax의 값을 대입한다.
```

mov는 대입이라 생각하면 편합니다.

mov 어셈블리어

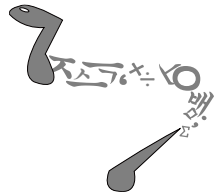
Negative.c

```
01  int main()
02  {
03      int var = 10;
04      var = -var;
05  }
```

디스어셈블리 (Pure/x86)

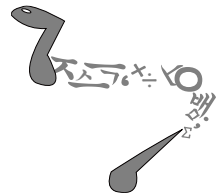
```
int main()
{
00401000  push        ebp
00401001  mov         ebp,esp
00401003  push        ecx
    int var = 10;
00401004  mov         dword ptr [var],0Ah
    var = -var;
0040100B  mov         eax,dword ptr [var]
0040100E  neg         eax
00401010  mov         dword ptr [var],eax
}
```

일단 오늘은 가볍게 mov만 해석해서 비교하시면 됩니다.



Chapter. • 5

Q&A



Q&A